Modalità di Esame

- Prova scritta su scripting e System Programming
- Progetto Linux (Architettura Client-Server)

Modalità di Esame

- Il “Progetto” consiste in:
  - Realizzazione di un software con allegata relazione
  - I progetti verranno assegnati verso la metà corso
  - Discussione sul software
    - Problematiche affrontate
    - Scelte implementative
    - Soluzioni particolari
    - ...

Modalità di Esame

- Il progetto viene assegnato a **gruppi** composti da **al più 2 studenti**.
  - Sono ammessi gruppi composti da un solo studente *solo in casi particolari (e.g., studenti lavoratori)*

- Tutti i membri del gruppo *devono discutere il progetto*
  - *Non possono discutere il progetto separatemente dai suoi.*
  - *Non siete obbligati a sostenere l'esame nello stesso appello*

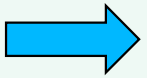
Modalità di Esame

- Il progetto deve essere consegnato *prima* della prova scritta
  - Con deroga per il primo appello (entro Marzo)
- Il voto finale valuta l'esito *dello scritto e dei progetti*
- Durante la discussione del progetto sono possibili altre domande

- Compilazione -

Per compilare in ambiente Unix, un programma C scritto nel file **source.c** si utilizza un compilatore GNU per sorgenti C, che viene invocato con il comando **gcc**:

Compilazione:

$ **gcc** source.c  **a.out** (eseguibile prodotto, formato ELF)

Opzioni comando gcc:

- flag "-o": $ gcc -o **eseguibile.out sorgente.c**
- flag "-v" : per sapere la versione del compilatore
- flag "-E": per invocare solo il preprocessore
- flag "-c": per creare il codice oggetto senza chiamare il linker
- falg "-On": ottimizzazione: O0, O1, O2 (standard), Os, Ofast
- flag "-Wall": per aumentare il livello dei messaggi prodotti in fase di compilazione

- $ man gcc (manuale in linea)

Messaggi del Compilatore

- Il compilatore invia dei messaggi all' operatore:
  - **Messaggi di avvertimento** (warning messagges)
    - I messaggi di avvertimento indicano la presenza di parti di codice presumibilmente mal scritte o di problemi che potrebbero avvenire in seguito, durante l'esecuzione del programma**.** I messaggi di avvertimento non interrompono comunque la compilazione***.***
  - **Messaggi d'errore** (error messagges)
    - I messaggi di errore invece indicano qualcosa che deve essere necessariamente corretto e causano l'interruzione della compilazione.

- Per **inibire** tutti i messaggi di **warning** usare l'opzione –w
  - $ gcc –w –o eseguibile.out source.c

- Per **aumentare** i **warning** usare l'opzione **–Wall** e **-Wextra**
  - $ gcc –Wall -Wextra –o eseguibile.out source.c

Funzione Principale C

Funzione principale in C

```
int main(void) {
    // nessun argomento
    return 0;
}
```

Oppure

```
int main(int argc, char *argv[]) {
    // argc = numero di argomenti (incluso il nome del programma)
    // argv = array di stringhe con gli argomenti
    return 0;
}
```

Valore di ritorno:

- 0 = successo
- diverso da 0 = errore

Funzione Principale C

Funzione principale in C e argomenti

```
#include <stdio.h>

int main(int argc, char *argv[]) {
    printf("Numero argomenti: %d\n", argc);
    for (int i = 0; i < argc; i++) {
        printf("argv[%d] = %s\n", i, argv[i]);
    }
    return 0;
}
```

Compilazione

```
gcc -o stampaArgomenti stampaArgomenti.c
```

Lancio

```
./stampaArgomenti uno due tre
```

```
Numero argomenti: 4
argv[0] = ./prog
argv[1] = uno
argv[2] = due
argv[3] = tre
```

Chiamata di Sistema

- Parametri inseriti nello stack dal programma e estratti dallo stack dal SO
- Programma in C invoca una chiamata di sistema:
  - Invocata da una procedura con il nome della sys call *read(fd, buffer, nbytes)*
    - Restituisce numero byte o -1 nel var globale errno

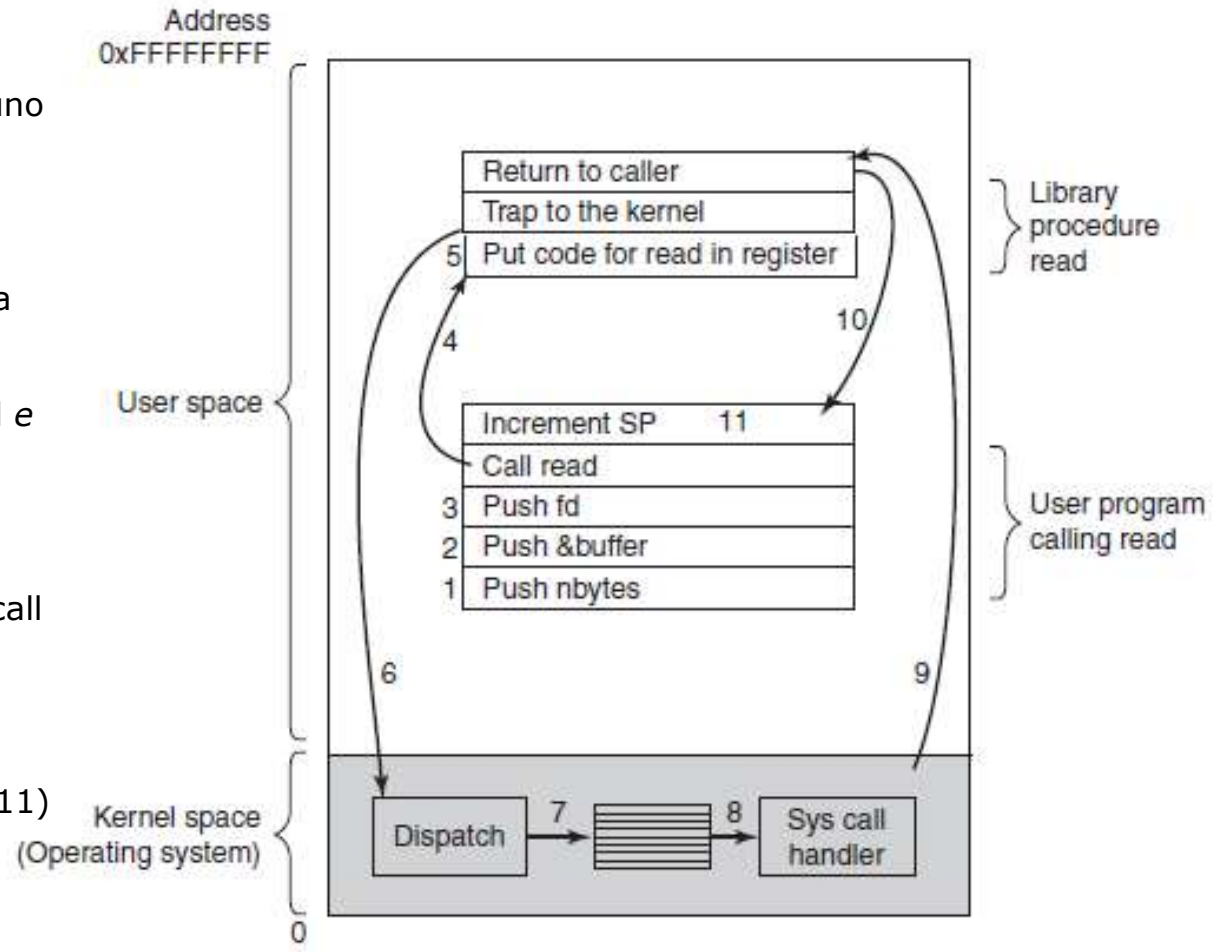
Il programma chiamante predispone la chiamata mettendo prima i paramentri in uno stack (passi 1–3) – il buffer passato per riferimento

La procedura chiama il read (passo 4) mettendo il codice della chiamata a sistema di read in un registro atteso dal Kernel

Quindi manda una istruzione *Trap* al kernel *e* passa il controllo al kernel in kernel mode (passi 5-6)

Il kernel trova il codice della chiamata e mediante una tabella trova l'handler della call (passi 7 e 8) finita l'esecuzione passa il risultato al chiamante (passo 9)

Il chiamante riprende il controllo in user mode (passo 10) e pulisce lo stack (passo 11) incrementando lo Stack Pointer (SP)

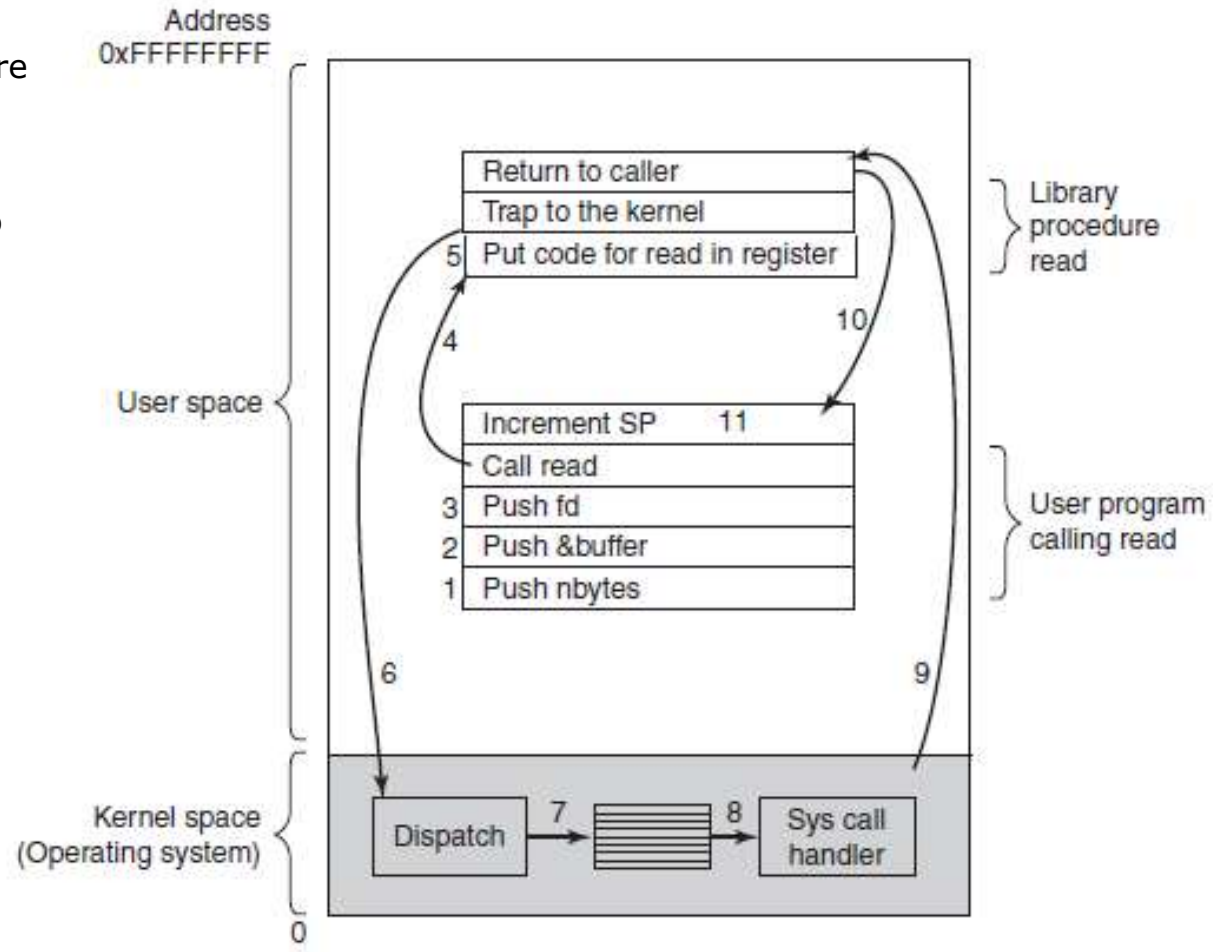


Chiamata di Sistema

- Parametri inseriti nello stack dal programma e estratti dallo stack dal SO
- Programma in C invoca una chiamata di sistema:
  - Invocata da una procedura con il nome della sys call *read(fd, buffer, nbytes)*
    - Restituisce numero byte o -1 nel var globale errno

Le procedure POSIX non si mappano sempre uno a uno con le system call offerte dal SO

Nella maggior parte dei casi chiamano effettivamente le sys call. Alcune eseguono operazioni senza fare il TRAP del kernel



I/O di basso livello

- La maggior parte delle operazioni sui file ordinari in ambiente UNIX si possono eseguire utilizzando solo le cinque chiamate di sistema open, read, write, lseek, close.
- Il kernel associa un *file descriptor ad ogni file aperto*
  - *Il file descriptor è un intero*
  - *Quando un file viene aperto con open, la funzione open restituisce il file descriptor associato al file*
- *Le costanti simboliche STDIN_FILENO (0), STDOUT_FILENO (1) e STDERR_FILENO (2) sono definite in unistd.h*

Descrittori di file

Alla richiesta di aprire un file esistente o di creare un nuovo file, il kernel ritorna un **descrittore di file** al processo chiamante

Quando si vuole **leggere o scrivere** su un file si passa come argomento a **read** e **write** il descrittore ritornato da **open**

Per convenzione il descrittore 0 viene associato allo standard input, 1 allo standard output e 2 allo standard error

I numeri 0, 1 e 2 possono essere sostituiti dalle costanti

STDIN_FILENO STDOUT_FILENO STDERR_FILENO

definite nell'header <unistd.h>

La funzione **open**

```
#include <sys/types.h>  /*data types*/
#include <sys/stat.h>   /*  data returned by stat()*/
#include <fcntl.h>      /*  file control options */

int open(const char *pathname, int oflag, ... /* mode_t mode */ );
```

- Restituisce il descrittore del file, -1 in caso di errore;
- Permette sia di aprire un file già esistente che di creare il file nel caso in cui questo non esista;
- pathname è il pathname (assoluto o relativo) del file;
- oflag permette di specificare le opzioni, mediante costanti definite in <fcntl.h>, combinate con “|” (or bit-a-bit);
- mode è il modo del file ed è un parametro opzionale, utilizzato solo nel caso di creazione del file (“…” modo ISO C per dire variabile).

Chiamata di sistema open

oflag può assumere diversi valori (definiti nell'header <fcntl.h>):

- O_RDONLY apri solo in lettura
- O_WRONLY apri solo in scrittura
- O_RDWR apri in lettura e scrittura

Solo una delle precedenti costanti può essere specificata, con una combinazione OR di:

- O_APPEND esegue un appen dalla fine del file per ciascuna write
- O_CREAT crea il file se non esiste
- O_EXCL se utilizzato insieme a O_CREAT, ritorna un errore se il file esiste
- O_TRUNC se file esiste e aperto con successo write only/read-write

I flag di **open**

O_RDONLY	Apre il file in sola lettura. Questa opzione non puo' essere combinata con O_WRONLY e O_RDWR.
O_WRONLY	Apre il file in sola scrittura. Questa opzione non puo' essere combinata con O_RDONLY e O_RDWR.
O_RDWR	Apre il file in lettura e scrittura. Questa opzione non puo' essere combinata con O_WRONLY e O_RDONLY.
O_APPEND	L'offset del file e' posto alla fine del file prima di ogni chiamata a write
O_CREAT	Crea il file se esso non esiste. Richiede che open sia utilizzata con l'argomento mode, per la specifica del modo del file.
O_EXCL	Genera un errore se e' stata specificata anche l'opzione O_CREAT e se il file gia' esiste.
O_TRUNC	Tronca il file a zero byte se esso esiste ed e' stato aperto in sola scrittura o in lettura-scrittura.
O_NOCTTY	Se pathname si riferisce ad un terminale, non lo alloca come terminale di controllo per il processo.
O_NONBLOCK	Se pathname si riferisce ad una FIFO o ad un file di dispositivo, definisce la modalita' nonblocking per l'apertura e l'I/O sul file.
O_SYNC	Specifica che ogni chiamata write deve attendere per il completamento dell'I/O sul dispositivo fisico.

I permessi per **open**

S_IRWXU	Permesso di lettura, scrittura ed esecuzione per il proprietario
S_IRUSR	Permesso di lettura per il proprietario
S_IWUSR	Permesso di scrittura per il proprietario
S_IXUSR	Permesso di esecuzione per il proprietario
S_IRWXG	Permesso di lettura, scrittura ed esecuzione per il gruppo
S_IRGRP	Permesso di lettura per il gruppo
S_IWGRP	Permesso di scrittura per il gruppo
S_IXGRP	Permesso di esecuzione per il gruppo
S_IRWXO	Permesso di lettura, scrittura ed esecuzione per gli altri
S_IROTH	Permesso di lettura per gli altri
S_IWOTH	Permesso di scrittura per gli altri
S_IXOTH	Permesso di esecuzione per gli altri

Esempi di open

- open(“prova.txt”, O_RDONLY)
- open(“prova.txt”, O_RDONLY | O_CREAT, S_IRWXU)
- open(“prova.txt”, O_RDWR | O_CREAT | O_EXCL, S_IRWXU)

Trattare gli errori

- Molte system call restituiscono -1 in caso di errore
- Per avere piu' informazioni, si usa la variabile globale ***errno*** (error number)
- La funzione ***perror***(const char *) stampa la stringa passata come parametro, e poi un messaggio in base al valore corrente di errno
- Esempi:

```
int fd = open(“prova.txt”, O_RDNLY);
if (fd<0) perror(“errore di open”);
```

```
int fd;
if ( (fd=open(“prova.txt”, O_RDNLY)) < 0)
        perror(“errore di open”);
```

close

#include <unistd.h>

int close(int filedes)

- Chiude il file identificato da filedes e precedentemente aperto con open
- Restituisce 0 in caso di successo o -1 in caso di errore

read

#include <unistd.h>

ssize_t read(int filedes, void *buf, size_t nbytes);

- Restituisce:
  - il numero di byte effettivamente letti
  - 0 se ci troviamo alla fine del file
  - -1 in caso di errore
- L'operazione di lettura avviene partendo dall'offset corrente del file
  - L'offset viene incrementato opportunamente

read

Il numero di byte letti può essere diverso dal paramentro nbytes quando:

- Il numero di byte ancora presenti nel file è inferiore ad nbytes.
- La lettura avviene da un terminale (si legge una riga alla volta).
- La lettura avviene da un buffer di rete (nbyte superiore)
- La lettura avviene da una pipe o una FIFO
- L'operazione e' interrotta da un segnale.

write

#include <unistd.h>

ssize_t write(int filedes, void *buff, size_t nbytes);

- Restituisce:
  - il numero di byte effettivamente scritti
  - -1 in caso di errore
- L'operazione di scrittura avviene partendo dall'offset corrente del file
  - L'offset viene incrementato opportunamente

Scambio dati: encoding

- Per scambiarsi i dati client e server devono fare encoding-decoding
  - Esempio conversione array-stringa

```
int a[] = {10, 20, 30, 40, 50};
int n = 5;
char msg[100] = "";
char temp[10];           // dichiarata fuori dal ciclo

for (int i = 0; i < n; i++) {
    sprintf(temp, "%d ", a[i]);  // converte numero in testo
    strcat(msg, temp);           // aggiunge al messaggio finale
}

write(sd, msg, strlen(msg));     // invia "10 20 30 40 50 "
```

Scambio dati: decoding

- Per scambiarsi i dati client e server devono fare encoding-decoding
  - Esempio: parsing e conversione stringa-array

```
char msg[100];
int vals[10];
int i = 0;

read(sd, msg, sizeof(msg) - 1);
msg[strcspn(msg, "\n")] = '\0';    // trova primo newline e chiude la stringa

char *token = strtok(msg, " ");    // divide la stringa in token delimitati da spazi
while (token != NULL) {
    vals[i++] = atoi(token);       // converte token in intero
    token = strtok(NULL, " ");     // passa al prossimo token
}
```

Scambio dati: binari

- Nel C, un socket trasmette byte, non tipi. Quindi dobbiamo definire noi come rappresentare l'array in byte.

- Problemi da gestire:
  - **Tipi a dimensione fissa:** usare uint32_t o int32_t da <stdint.h>
  - **Ordine dei byte:** usare *network byte order* (htonl() / ntohl())
  - **Framing:** indicare quanti elementi si mandano (così il ricevente sa dove finisce)

Scambio dati: binari

- Nel C, un socket trasmette byte, non tipi. Quindi dobbiamo definire noi come rappresentare l'array in byte.

- Client invia n dati di un array

```
uint32_t vals[] = {10, 20, 30, 40, 50};
    uint32_t n = sizeof(vals) / sizeof(vals[0]);

    // --- Invio dimensione ---
    uint32_t n_net = htonl(n);
    write(sd, &n_net, sizeof(n_net));

    // --- Invio valori convertiti ---
    for (uint32_t i = 0; i < n; i++) {
        uint32_t v_net = htonl(vals[i]);
        write(sd, &v_net, sizeof(v_net));
    }
```

Scambio dati: binari

- Nel C, un socket trasmette byte, non tipi. Quindi dobbiamo definire noi come rappresentare l'array in byte.

- Server legge n dati di un array

```
// --- Legge prima la dimensione ---
    uint32_t n_net, n;
    read(sd, &n_net, sizeof(n_net));
    n = ntohl(n_net);  // converte in ordine host

    printf("Server: mi aspetto %u numeri\n", n);
    uint32_t *vals = malloc(n * sizeof(uint32_t));

    // --- Legge gli elementi uno a uno ---
    for (uint32_t i = 0; i < n; i++) {
        uint32_t v_net;
        read(sd, &v_net, sizeof(v_net));
        vals[i] = ntohl(v_net);
    }
```

Socket TCP

- Indirizzi

in netinet/ip.h:

```
struct sockaddr_in {
        sa_family_t     sin_family;
        u_int16_t       sin_port;
        struct in_addr  sin_addr;
};

struct in_addr {
        u_int32_t       s_addr;
};
```

vedere: man 7 ip

AF_INET

porta, in network order

indirizzo IP, in network order

Socket TCP

- Indirizzi

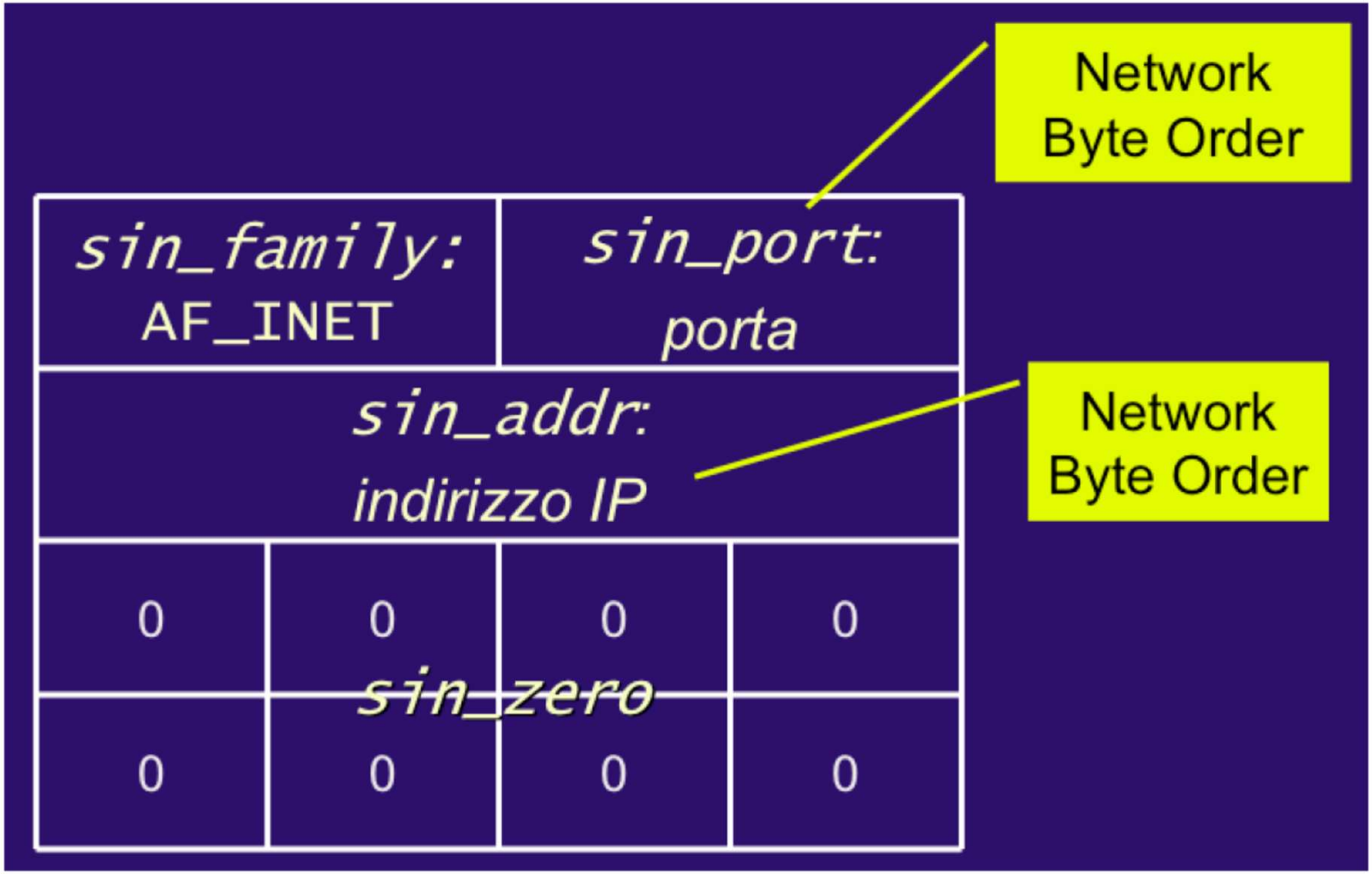
- Siamo interessati a `sa_family = AF_INET`, cioè al dominio “internet”
- In tal caso servono 2 byte per un numero di porta e 4 byte per un indirizzo IP

```
#include <netinet/in.h>
struct sockaddr_in {
    short int sin_family;          // Address family
    unsigned short int sin_port;   // Port number
    struct in_addr sin_addr;       // IP Internet Address
    unsigned char sin_zero[8];     // Stessi byte di…
};                                 // …struct sockaddr
```

14 bytes

Socket TCP

- Indirizzi



Socket TCP

- Indirizzi

  - Identificati da:

    1) un indirizzo IP (intero a 32 bit, es. 143.225.5.3)

    2) una porta (intero a 16 bit, da 0 a 65535)

  - Porte:
    - per offrire diversi servizi dallo stesso indirizzo IP
    - da 0 a 1023: porte riservate (ai processi di root)
    - da 5000 a 32768: porte utente
    - altre: porte effimere (per i client, ai quali non interessa scegliere una porta specifica)

Socket TCP

- Indirizzi

  - Identificati da:

    1) un indirizzo IP (intero a 32 bit, es. 143.225.5.3)

    2) una porta (intero a 16 bit, da 0 a 65535)

  - Porte:
    - per offrire diversi servizi dallo stesso indirizzo IP
    - da 0 a 1023: porte riservate (ai processi di root)
    - da 5000 a 32768: porte utente
    - altre: porte effimere (per i client, ai quali non interessa scegliere una porta specifica)

Socket TCP

- Esempi di porte riservate

  - 21 ftp (trasferimento file)
  - 22 ssh (login remoto sicuro)
  - 25 smtp (invio email)
  - 79 finger (informazioni sugli utenti)
  - 80 http (web)
  - 143 imap (lettura email)

  - Lista ufficiale su: http://www.iana.org/

Socket TCP

- Rappresentazione dati in memoria

  - Come viene memorizzato un intero in memoria?
    - sono usati 4 bytes
    - consideriamo 65537, cioe' $2^{16} + 1$, all'indirizzo 100
    - codifica “Big Endian” (prima il byte *piu'* significativo):

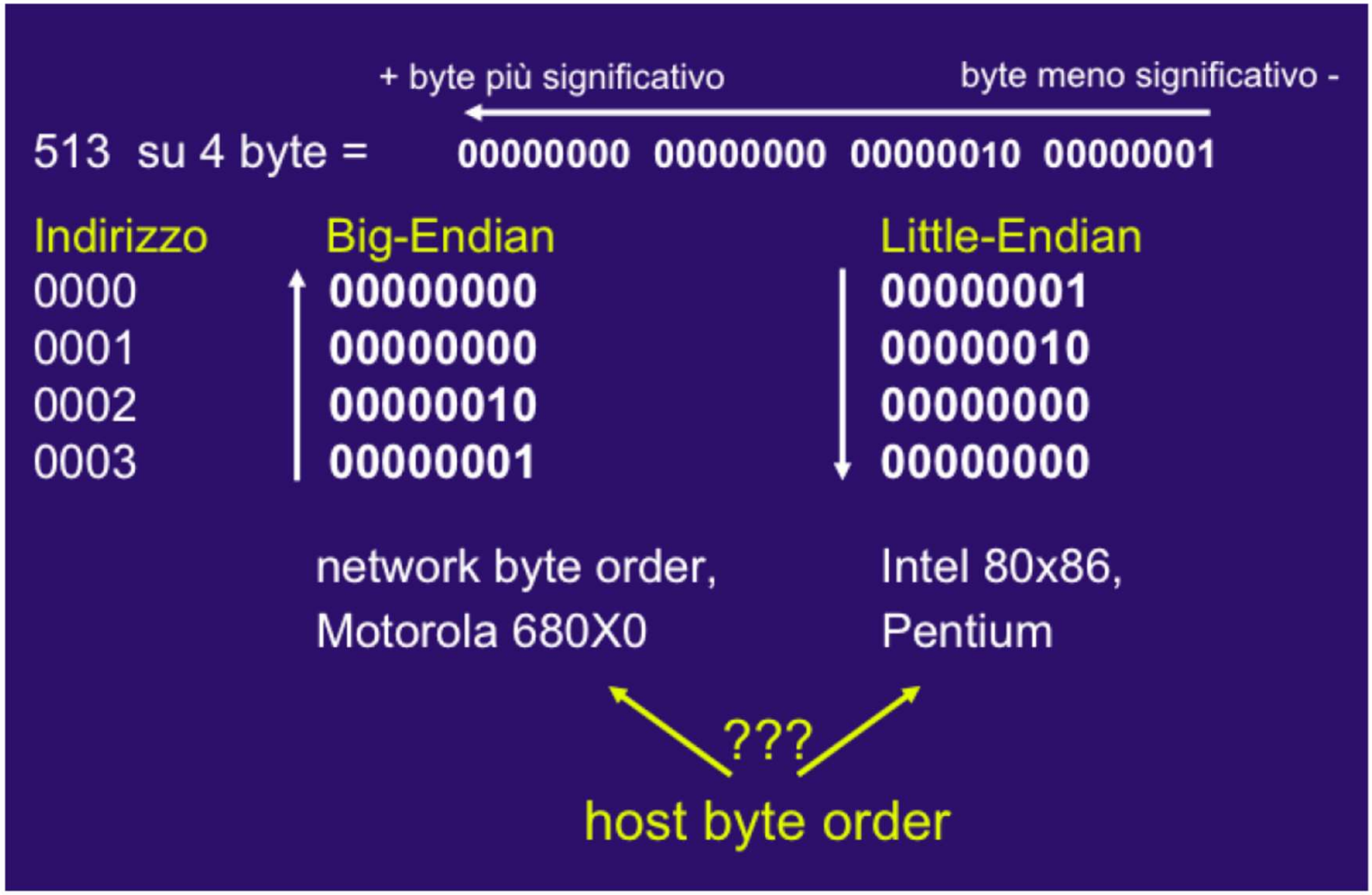
      00000000 00000000 10000000 00000001
      100 101 102 103

    - codifica “Little Endian” (prima il byte *meno* significativo):

      00000001 10000000 00000000 00000000
      100 101 102 103

Socket TCP

- Rappresentazione dati in memoria



- BE: network byte order (TCP/IP), più raro come host order, es. PowerPC, SPARC
- LE: x86-64, ARM, RISC-V

Socket TCP

- Rappresentazione dati in memoria

  - i processori possono essere di entrambi i tipi
    - i protocolli TCP/IP lavorano con **big endian**
    - sono necessarie funzioni di **conversione**

  - conversioni tra unsigned:
    - #include <netinet/in.h>
    - uint32_t **h**to**n**l(uint32_t x)
    - uint16_t **h**to**n**s(uint16_t x)
    - uint32_t **n**to**h**l(uint32_t x)
    - uint16_t **n**to**h**s(uint16_t x)

h	= Host
n	= Network (big endian)
l	= Long (4 bytes)
s	= Short (2 bytes)

Socket TCP

- Il server chiama il bind per agganciarsi all'indirizzo
- Versione IPv4
  - INADDR_ANY per ascoltare da qualunque indirizzo: #define che vale 0.0.0.0

```
#include <netinet/in.h>
#include <arpa/inet.h>

struct sockaddr_in mio_indirizzo;

// azzera tutto
memset(&mio_indirizzo, 0, sizeof(mio_indirizzo));

mio_indirizzo.sin_family = AF_INET;
mio_indirizzo.sin_port   = htons(5200);      // network byte order
mio_indirizzo.sin_addr.s_addr = htonl(INADDR_ANY);

if (bind(fd, (struct sockaddr *)&mio_indirizzo, sizeof(mio_indirizzo)) < 0) {
    perror("bind");
    exit(1);
}
```

Socket TCP

- Il client chiama una connect
- Versione IPv4

```
#include <netinet/in.h>   // sockaddr_in, htons
#include <arpa/inet.h>    // inet_pton
#include <string.h>       // memset
#include <stdio.h>        // perror
#include <stdlib.h>       // exit

/* ...creazione socket => fd ... */

struct sockaddr_in indirizzo;
memset(&indirizzo, 0, sizeof(indirizzo)); // azzera la struttura

indirizzo.sin_family = AF_INET;
indirizzo.sin_port = htons(5200);

if (inet_pton(AF_INET, "143.225.5.3", &indirizzo.sin_addr) != 1) {
    perror("inet_pton"); exit(1);}

if (connect(fd, (struct sockaddr *)&indirizzo, sizeof(indirizzo)) < 0) {
    perror("connect"); exit(1);
}
```

Socket TCP

- Il server si mette in ascolto

int **listen**(int **sockfd**, int **lunghezza_coda**);

- Mette il socket in modalita' di ascolto
  - in attesa di nuove connessioni
- Solo per socket SOCK_STREAM e SOCK_SEQPACKET
- Il secondo argomento specifica quante connessioni possono essere in attesa di essere accettate
  - Se il numero di connessioni in attesa supera il secondo parametro, il client riceve “connection refused”
- Restituisce 0 oppure -1

Socket TCP

- Il server accetta nuove connessioni

```
int accept(int sockfd,
        struct sockaddr *indirizzo_client,
        socklen_t *dimensione_indirizzo);
```

- Il secondo e terzo argomento servono ad identificare il client
  - possono essere NULL
- Restituisce un nuovo descrittore! (oppure -1)
  - crea un nuovo socket, dedicato a questa nuova connessione
  - il vecchio socket resta in ascolto
- Blocca il processo se non vi sono connessioni in attesa
  - Il socket puo' essere marcato non-blocking.

Socket TCP

- Struttura di un server

```
int fd1, fd2;
struct sockaddr_in mio_indirizzo = {0};

mio_indirizzo.sin_family      = AF_INET;
mio_indirizzo.sin_port        = htons(5200);
mio_indirizzo.sin_addr.s_addr = htonl(INADDR_ANY);

fd1 = socket(AF_INET, SOCK_STREAM, 0);

bind(fd1, (struct sockaddr *)&mio_indirizzo, sizeof(mio_indirizzo));
listen(fd1, 5);

fd2 = accept(fd1, NULL, NULL);
/* ... */
close(fd2);
close(fd1);
```

Socket TCP

- Struttura di un server

```
#include <netinet/in.h>
#include <arpa/inet.h>
#include <string.h>
#include <unistd.h>     // close

int main(void) {
    int s, c;
    struct sockaddr_in addr = {0};

    addr.sin_family      = AF_INET;
    addr.sin_port        = htons(5200);
    addr.sin_addr.s_addr = htonl(INADDR_ANY);

    s = socket(AF_INET, SOCK_STREAM, 0);
    bind(s, (struct sockaddr *)&addr, sizeof(addr));
    listen(s, 5);
    c = accept(s, NULL, NULL);
    /* ... comunico con c ... */
    close(c); close(s);
}
```

Socket TCP

- Struttura di un client

```
#include <netinet/in.h>
#include <arpa/inet.h>
#include <string.h>
#include <unistd.h>     // close

int main(void) {
    int s;
    struct sockaddr_in addr = {0};

    addr.sin_family = AF_INET;
    addr.sin_port   = htons(5200);

    inet_pton(AF_INET, "127.0.0.1", &addr.sin_addr);

    s = socket(AF_INET, SOCK_STREAM, 0);
    connect(s, (struct sockaddr *)&addr, sizeof(addr));

    /* ... comunico con s ... */
    close(s);
}
```

Socket TCP

- Scrittura con write

```
#include <unistd.h>
ssize_t write( int fd,
               const void *buf,
               size_t count);
```

- invia il contenuto del buffer `buf` al socket specificato
- si usa esclusivamente con `SOCK_STREAM`
- restituisce il numero di byte inviati oppure -1 in caso di errore
- è la stessa funzione che consente la scrittura su file

Socket TCP

- Ricezione con read

```
#include <unistd.h>
ssize_t read( int fd,
              void *buf,
              size_t count);
```

- solo per socket connessi (SOCK_STREAM)
- legge un messaggio di lunghezza massima `len` dal socket
- se non c'è alcun messaggio, il programma rimane sospeso
  - chiamata *bloccante*
- la funzione ritorna il numero di byte letti, `-1` in caso di errore
- è la stessa funzione che consente la lettura da un file

Socket TCP

- Lettura scrittura

Stessa interfaccia per scrittura su file:

- Si possono passare le sd a processi figli che possono non distingure tra sd e fd

- Possono essere usati anche con processi che lavorano su file locali.

- Esistono altre funzioni specifiche per leggere e scrivere su socket: 3 modalità di send, 3 modalità di recieve.

Socket TCP

- Lettura scrittura

Send & Recv:

- **send** è come write (richiede connessione stabilita), ma ha flag per specificare modalità di scrittura,
- **sendto** permette la scrittura su connectionless socket,
- **sendmsg** per specificare buffer multipli
- **recv** come read con flag,
- **recvfrom** per ottenere indirizzo della fonte,
- **rcvmsg** per ricevere da buffer multipli

Socket TCP

- Lettura scrittura

```
#include <sys/types.h>
#include <sys/socket.h>
int send(int s,
         const void *msg, int len,
         unsigned int flags);
int sendto( int s, const void *msg, int len,
            unsigned int flags,
            const struct sockaddr *to, int tolen);
```

man 2 send

- `send()` può essere utilizzata solo se `s` è stato connesso; `sendto()` sempre perché richiede di specificare l'indirizzo di destinazione
- restituisce `-1` in caso di errore oppure il numero di byte effettivamente trasmessi
- `flags` si può lasciare a zero

Socket TCP

- Lettura scrittura

```
#include <sys/types.h>
#include <sys/socket.h>
int recv(int s,
         void *buf, int len,
         unsigned int flags);
int recvfrom(int s, void *buf, int len,
             unsigned int flags,
             struct sockaddr *from,
             int *fromlen);
```

man 2 recv

- ricevono in `buf` non più di `len` byte. Se `from` non è `NULL`, la struttura `sockaddr` verrà riempita con l'indirizzo del mittente
- restituisce `-1` in caso di errore oppure il numero di byte effettivamente ricevuti
- `flags` si può lasciare a zero

Socket TCP

- Lettura scrittura

```
#include <sys/types.h>
#include <sys/socket.h>
int recvfrom(int s, void *buf, int len,
             unsigned int flags,
             struct sockaddr *from,
             int *fromlen);
```

man 2 recv

- ricevono in `buf` non più di `len` byte. Se `from` non è `NULL`, la struttura `sockaddr` verrà riempita con l'indirizzo del mittente
- restituisce `-1` in caso di errore oppure il numero di byte effettivamente ricevuti
- se non arriva alcun messaggio, il programma rimane sospeso (la chiamata è "bloccante")
- `flags` si può lasciare a zero

Socket TCP

- **Send/Receive**
  - accettano un parametro “flags” che permette comportamenti extra rispetto a write/read:

```
ssize_t send(int sock, const void *buf, size_t len, int flags);
ssize_t recv(int sock, void *buf, size_t len, int flags);
```

flag	**effetto**
`MSG_DONTWAIT`	non blocca
`MSG_PEEK`	legge senza consumare receive queue
`MSG_WAITALL`	aspetta finché il len non è raggiunto

  - MSG_DONTWAIT su recv() non aspetta il dato (es. polling)
  - MSG_DONTWAIT su send() non aspetta se il buffer è pieno
  - MSG_PEEK su recv() per testare un dato prima di rimuoverlo
  - MSG_WAITALL su recv() aspetta che il len sia raggiunto
  - MSG_NOSIGNAL su send() evita il SIGPIPE

  - Senza flag stesso comportamento di read/write

Socket TCP

- **Send/Receive**
  - accettano un parametro “flags” che permette comportamenti extra rispetto a write/read:

```
ssize_t send(int sock, const void *buf, size_t len, int flags);
ssize_t recv(int sock, void *buf, size_t len, int flags);
```

flag	**effetto**
`MSG_DONTWAIT`	non blocca
`MSG_PEEK`	legge senza consumare receive queue
`MSG_WAITALL`	aspetta finché il len non è raggiunto

  - MSG_DONTWAIT su recv() non aspetta il dato (es. polling)
  - MSG_DONTWAIT su send() non aspetta se il buffer è pieno
  - MSG_PEEK su recv() per testare un dato prima di rimuoverlo
  - MSG_WAITALL su recv() aspetta che il len sia raggiunto
  - MSG_NOSIGNAL su send() evita il SIGPIPE

  - Senza flag stesso comportamento di read/write

Socket TCP

- **Esempio**
  - Server incrementa un numero passato dal client

```
#include <netinet/in.h>
#include <arpa/inet.h>
#include <unistd.h>
#include <signal.h>
#include <stdint.h>
#include <stdio.h>
#include <string.h>

static int s = -1;    // listening socket

void handler(int sig) {
    (void)sig;
    if (s >= 0) {
        shutdown(s, SHUT_RDWR);   // forza accept a ritornare
    }
}
```

Socket TCP

- **Esempio**
  - Server incrementa un numero passato dal client

```
int main(void) {

    signal(SIGINT, handler);
    s = socket(AF_INET, SOCK_STREAM, 0);

    struct sockaddr_in addr = {0};
    addr.sin_family      = AF_INET;
    addr.sin_port        = htons(5201);
    addr.sin_addr.s_addr = htonl(INADDR_ANY);
    bind(s, (struct sockaddr*)&addr, sizeof(addr));
    listen(s, 1);

    printf("[server] in ascolto (Ctrl-C per uscire)\n");

    while (1) {
        int c = accept(s, NULL, NULL);
        if (c < 0) {
            // probabile uscita causata da shutdown()
            break;
        }
```

Socket TCP

- **Esempio**
  - Server incrementa un numero passato dal client

```
uint32_t net_x;
        ssize_t r = recv(c, &net_x, sizeof(net_x), 0);

        if (r == sizeof(net_x)) {
            uint32_t x = ntohl(net_x);
            uint32_t next = x + 1;
            uint32_t net_next = htonl(next);
            send(c, &net_next, sizeof(net_next), 0);
        }

        close(c);
    }

    close(s);
    return 0;
}
```

Socket TCP

- **Esempio**
  - Client passa un numero

```
int main(void) {

    int s = socket(AF_INET, SOCK_STREAM, 0);

    struct sockaddr_in srv = {0};
    srv.sin_family = AF_INET;
    srv.sin_port   = htons(5201);
    inet_pton(AF_INET, "127.0.0.1", &srv.sin_addr);

    connect(s, (struct sockaddr*)&srv, sizeof(srv));

    int x;
    printf("Inserisci un numero: ");

    if (scanf("%d", &x) != 1) {
        printf("Input non valido.\n");
        close(s);
        return 1;
    }
```

Socket TCP

- **Esempio**
  - Client passa un numero

```
uint32_t net_x = htonl((uint32_t)x);
   send(s, &net_x, sizeof(net_x), 0);

   uint32_t net_res;
   ssize_t r = recv(s, &net_res, sizeof(net_res), 0);

   if (r == sizeof(net_res)) {
       uint32_t res = ntohl(net_res);
       printf("Risposta dal server: %u\n", res);
   } else {
       printf("Errore durante recv\n");
   }

   close(s);
   return 0;
}
```

Buffer output

- Le socket TCP hanno un buffer per l'output (send buffer) in cui vengono collocati temporaneamente i dati da trasmettere
- La dimensione di questo buffer può essere configurata mediante l'opzione SO_SNDBUF
- Se si chiama write() con n bite sul socket TCP, il kernel cerca di copiare n byte sul buffer del socket. Se il buffer del socket e' più piccolo di n byte o parzialmente occupato da dati non ancora trasmessi verranno copiati un numero minore di byte
- Se il socket è bloccante alla fine della write saranno inviati i byte indicati dal valore di ritorno della write

Socket

- **Lettura e scrittura safe**

```
#include <sys/types.h>
#include <sys/socket.h>
#include <errno.h>

ssize_t recv_all(int fd, void *buf, size_t n) {
    size_t received = 0;
    char *p = buf;

    while (received < n) {
        ssize_t r = recv(fd, p + received, n - received, 0);

        if (r < 0) {
            if (errno == EINTR)
                continue;     // riprova
            return -1;        // errore
        }
        if (r == 0) {
            // connessione chiusa -> restituisco quanti byte ho letto
            return received;
        }
        received += (size_t)r;
    }
    return (ssize_t)received;   // = n
}
```

Socket

- **Lettura e scrittura safe**

```
#include <sys/types.h>
#include <sys/socket.h>
#include <errno.h>

ssize_t send_all(int fd, const void *buf, size_t n) {
    size_t sent = 0;
    const char *p = buf;

    while (sent < n) {
        ssize_t w = send(fd, p + sent, n - sent, 0);

        if (w < 0) {
            if (errno == EINTR)
                continue;     // riprova
            return -1;        // errore
        }

        sent += (size_t)w;
    }

    return (ssize_t)sent;   // = n
}
```

Socket

- **Opzioni socket**
  - Opzione SO_REUSEADDR

```
int getsockopt(int fd, level, option, void *val, socklen_t len);
int setsockopt(int fd, level, option, void *val, socklen_t *len);
```

- opzione di livello socket
- permette a bind di assegnare un indirizzo ancora occupato
- val deve puntare ad un intero
  - se l'intero è diverso da zero, questa opzione è attiva
  - se l'intero puntato contiene zero, l'opzione è inattiva
  - len è pari a sizeof(int)

Socket

- **Opzioni socket**
  - Opzione SO_REUSEADDR
- **Contesto:**
  - Quando una connessione TCP si chiude, non viene smontata subito: resta in TIME_WAIT per ~1–4 minuti. Serve per:
    - evitare pacchetti “ritardati” che possano confondere una nuova connessione garantire chiusura ordinata
    - Durante TIME_WAIT, un nuovo socket NON può bindare quella porta → **ma con SO_REUSEADDR sì.**
- **Quando serve:**
  - Dopo uso occore ancora bind(sock, ...) su stesso indirizzo
  - il server non riesce a ripartire → “address already in use”
  - Con l’opzione: parte senza problemi

Funzioni bloccanti

- La funzione accept() e le funzioni per gestire I/O sono bloccanti (read, recv)
- Il server ricorsivo tradizionale
  - attende su accept() una connessione
  - Quando accetta una connessione il processo/thread principale crea un processo/thread dedicato per il controllo e si mette in attesa di altro
- Alternative:
  - Usare opzioni per le socket per impostare un timeout
  - Usare socket non bloccante con funzione fcntl (funzione di gestione fd)

Socket

- **Opzioni receive_all (con possibilità di timeout)**

```
ssize_t recv_all(int fd, void *buf, size_t n) {
    size_t received = 0; char *p = (char *)buf;

    while (received < n) {
        ssize_t r = recv(fd, p + received, n - received, 0);
        if (r > 0) {
            received += (size_t)r;
            continue;
        }
        if (r == 0) {
             return (ssize_t)received; // EOF: peer ha chiuso
        }
        if (errno == EINTR) // r < 0
            continue; // interrotto da segnale: riprova
        if (errno == EAGAIN || errno == EWOULDBLOCK)
            return -2; // timeout / would-block
        return -1;      // altro errore
    }
    return (ssize_t)received;   // == n
}
```

Socket

- **Opzioni send_all (con possibilità di timeout)**

```
ssize_t send_all(int fd, const void *buf, size_t n) {
    size_t sent = 0; const char *p = (const char *)buf;

    while (sent < n) {
        ssize_t w = send(fd, p + sent, n - sent, 0);
        if (w > 0) {
            sent += (size_t)w;
            continue;
        }
        if (w == 0) {
             return -1; // Raro per send()
        }
        if (errno == EINTR) // w < 0
            continue; // interrotto da segnale: riprova
        if (errno == EAGAIN || errno == EWOULDBLOCK)
            return -2; // non può scrivere ora (timeout / non-blocking)
        return -1;      // altro errore
    }
    return (ssize_t)sent;   // == n
}
```

Buffer output/input

- Le socket TCP hanno un buffer per l'output e per l'input in cui vengono collocati temporaneamente i dati da trasmettere
- La dimensione di questo buffer può essere configurata mediante l'opzione SO_SNDBUF, SO_RCVBUF
- Se si chiama **write()/send()** con n bite sul socket TCP, il kernel:
  - cerca di copiare n byte sul send buffer del socket.
  - Se il buffer del socket e' più piccolo di n byte o parzialmente occupato da dati non ancora trasmessi verranno copiati un numero minore di byte (short write)
  - se la socket è bloccante, può aspettare che si liberi spazio e poi copiare
  - se la socket è non-bloccante, ritorna EAGAIN se non c'è spazio

Buffer output/input

- Le socket TCP hanno un buffer per l'output e per l'input in cui vengono collocati temporaneamente i dati da trasmettere
- La dimensione di questo buffer può essere configurata mediante l'opzione SO_SNDBUF, SO_RCVBUF
- Chiamando **read()/receive()** con n bite sul socket TCP:
  - Copia n byte dal receive buffer (kernel) del socket al buffer dell'app
  - Può leggere un numero minore di byte (short read)
  - Se bloccante si blocca solo se non ci sono byte disponibili, altrimenti restituisce quelli presenti fino agli n richiesti
  - Se non bloccante restituisce subito quelli sul buffer del kernel, se nessun dato dà -1 (errore) con errno = EAGAIN (o EWOULBLOCK)

Socket

- **Socket non bloccante**
  - setsockopt() non c'è POSIX

- **Metodo comune: POSIX**
  - Si modifica il file descriptor con fcntl

```
#include <fcntl.h>

int flags = fcntl(fd, F_GETFL, 0);
fcntl(fd, F_SETFL, flags | O_NONBLOCK);
```

- **Effetto:**
  - recv() non blocca → restituisce -1, errno = EAGAIN se non ci sono dati
  - send() non blocca → restituisce -1, errno = EAGAIN se non c'è spazio
  - funziona su TCP/UDP/pipe, ecc.

sd è il socket precedentemente aperto; F_SETFL significa che si vuole impostare il *file status flag* al valore dall'ultimo parametro; O_NONBLOCK costante corrispondente al valore che significa non bloccante.

Socket

- **Socket non bloccante**
  - setsockopt() non c'è POSIX

- **Metodo comune: POSIX**
  - Si modifica il file descriptor con fcntl

```
#include <fcntl.h>

int flags = fcntl(fd, F_GETFL, 0);
fcntl(fd, F_SETFL, flags | O_NONBLOCK);
```

- **Effetto:**
  - recv() non blocca → restituisce -1, errno = EAGAIN se non ci sono dati
  - send() non blocca → restituisce -1, errno = EAGAIN se non c'è spazio
  - funziona su TCP/UDP/pipe, ecc.

Socket

- **Socket non bloccante**
  - setsockopt() non c'è POSIX
- **Esempio**

```
int fd = socket(AF_INET, SOCK_STREAM, 0);
    if (fd < 0) {
        perror("socket");
        return 1;
    }

    /* ---- Rendo la socket NON BLOCCANTE ---- */
    int flags = fcntl(fd, F_GETFL, 0);
    if (flags == -1) {
        perror("fcntl(F_GETFL)");
        close(fd);
        return 1;
    }
    if (fcntl(fd, F_SETFL, flags | O_NONBLOCK) < 0)
{
        perror("fcntl(F_SETFL)");
        close(fd);
        return 1;
    }
```

Socket

- **Gestione interruzione**
  - send() o recv() possono essere interrotte
  - EINTR vale per tutte le syscall bloccanti (interrotte da segnale ritornano -1 e EINTR)

- **Perché su read/write su file non gestite:**
  - Molte libc ri-eseguono la read direttamente
  - Su socket, pipe, sleep, accept, etc. va gestita

- **Con SA_RESTART**
  - Syscall bloccante interrotta da segnale viene rieseguita invece di -1 ed EINTR
  - Altrimenti va gestita a mano

```
while (read(...) < 0 && errno == EINTR) continue;
```

- **Non sempre SA_RESTART va bene**
  - select(), waited(), nanosleep() non funziona correttamente

Socket

- **Gestione interruzione**
  - send() o recv() possono essere interrotte
  - EINTR vale per tutte le syscall bloccanti (interrotte da segnale ritornano -1 e EINTR)

- **Gestione di SA_RESTART**
  - Solo sigaction() permette di impostare SA_RESTART

```
struct sigaction {
    void     (*sa_handler)(int);
    sigset_t   sa_mask;
    int        sa_flags;   // <—— contiene SA_RESTART
};

struct sigaction sa = {0};
sa.sa_handler = mio_handler;
sa.sa_flags   = SA_RESTART;
sigaction(SIGUSR1, &sa, NULL);
```

Socket

- **Gestione interruzione**
  - send() o recv() possono essere interrotte
  - EINTR vale per tutte le syscall bloccanti (interrotte da segnale ritornano -1 e EINTR)

- **Gestione di accept senza SA_RESTART**
  - Abbiamo: accept() → segnale → handler → ritorno → accept() fallisce
  - → -1, errno = EINTR
  - Quindi impostata manualmente:

```
for (;;) {
    int sd = accept(listen_sd, ...);
    if (sd >= 0) break;
    if (errno == EINTR) continue;
    perror("accept");
}
```

Socket

- **Gestione interruzione**
  - send() o recv() possono essere interrotte
  - EINTR vale per tutte le syscall bloccanti (interrotte da segnale ritornano -1 e EINTR)

- **Gestione di accept con SA_RESTART**
  - Abbiamo: accept() → segnale → handler → ritorno → accept() viene riavviata
  - Quindi in Linux riavviata in automatico, ma non POSIX
  - Impostare:

```
struct sigaction sa;
memset(&sa, 0, sizeof(sa));
sa.sa_handler = mio_handler;
sa.sa_flags   = SA_RESTART;
sigemptyset(&sa.sa_mask);

sigaction(SIGINT, &sa, NULL);
```

Socket

- **Gestione SIGPIPE**
  - Quando viene chiusa una socket chi scrive riceve un SIGPIPE
  - Si può gestire ignorando + errno
  - Oppure si può gestire con handler di segna,e

- **Metodo:**

```
signal(SIGPIPE, SIG_IGN);

ssize_t w = send(sd, buf, len, 0);
if (w < 0) {
    if (errno == EPIPE) {
        // peer chiuso
        close(sd);
    }
}
```

- **Comportamento:**
  - In questo caso ignorato
  - Si gestisce con errno nel caso di write/signal
  - Nel caso di read/send, se legge 0 byte la connessione è interrotta

Socket

- **Gestione SIGPIPE**
  - Quando viene chiusa una socket chi scrive riceve un SIGPIPE
  - Si può gestire ignorando + errno
  - Oppure si può gestire con handler di segna,e

- **Metodo:**

```
signal(SIGPIPE, SIG_IGN);

ssize_t n = recv(sd, buf, sizeof(buf), 0);
if (n == 0) {
        // Il peer ha chiuso la connessione (FIN)
        printf("peer ha chiuso la connessione\n");
        close();
    }
```

- **Comportamento:**
  - In questo caso ignorato
  - Si gestisce con errno nel caso di write/signal
  - Nel caso di read/send, se legge 0 byte la connessione è interrotta

Socket

- **Gestione SIGPIPE**
  - Quando viene chiusa una socket chi scrive riceve un SIGPIPE
  - Si può gestire ignorando + errno
  - Oppure si può gestire con handler di segnale

- **Metodo:**

```
if (pthread_key_create(&key_sd, NULL) != 0) {
        perror("pthread_key_create");
        return 1;
    }

if (signal(SIGPIPE, sigpipe_handler) == SIG_ERR) {
        perror("signal");
        return 1;
    }
```

- **Comportamento:**
  - In questo caso ogni thread conserva la sua chiave privata
  - Quando scatta il segnale, il gestore prende la chiave e chiude
  - Attenzione a doppie chiusure (comportamento non definito)

Socket

- **Gestione SIGPIPE**
  - Quando viene chiusa una socket chi scrive riceve un SIGPIPE
  - Si può gestire ignorando + errno
  - Oppure si può gestire con handler di segnale

- **Metodo:**

```
static void sigpipe_handler(int signo)
{
    (void)signo;

    // Recupera il socket associato al thread attuale
    int sd = (int)(intptr_t)pthread_getspecific(key_sd);
    if (sd >= 0) {
        close(sd);
        // Evito doppi close, usa int → intptr_t → void*
        pthread_setspecific(key_sd, (void*)(intptr_t)-1);
    }

    const char msg[] = "SIGPIPE: chiuso socket thread\n";
    write(STDERR_FILENO, msg, sizeof(msg)-1);
}
```

Select

- select() permette di:
  - Attendere evento I/O su più socket contemporaneamente senza bloccarne uno mentre gli altri hanno dati pronti
  - Permette di evitare un thread per socket
  - Utile per gestire molti client in modo single-threaded

Select

int select (int numfds, fd_set *readfds, fd_set *writefds, fd_set *exceptfds,
struct timeval *timeout);

Header file sys/time.h, sys/types.h, unistd.h

- Controlla contemporaneamente lo stato di uno o più descrittori specificati
- Si blocca finché:
  - non avviene un'attività (lettura/scrittura) su un descrittore degli insiemi
  - non viene generata un'eccezione
  - non scade un timeout
- Restituisce
  - -1 in caso di errore
  - 0 se il timeout è scaduto
  - Altrimenti, il numero totale di descrittori

Parametri Select

Insiemi di descrittori da controllare

– **readfds**: pronti per operazioni di lettura

  Es: socket pronto per la lettura se c'è una connessione in attesa di accept()

– **writefds**: pronti per operazioni di scrittura

– **exceptfds**: exceptional conditions, quasi sempre nullo

readfds, writefds e exceptfds sono puntatori a variabili di tipo fd_set

– fd_set insieme dei descrittori (è una bit mask implementata con un array di interi)

numfds è il numero/range massimo di descrittori controllati da select()

– **ATTENZIONE**: Se maxd è il massimo descrittore usato, numfds = maxd + 1

– Può essere posto uguale alla costante FD_SETSIZE

Timeout

- timeout specifica il valore massimo che la funzione select() attende per individuare un descrittore pronto

```
struct timeval {
    int tv_sec;     // seconds
    int tv_usec;    // microseconds
};
```

- Se impostato a NULL (timeout == NULL)
  - select si blocca indefinitamente fino a quando è pronto un descrittore
- Se impostato a zero (timeout->tv_sec == 0 && timeout->tv_usec == 0 })
  - select non attende (da usare per polling dei descrittori senza bloccare)
- Se diverso da zero (timeout->tv_sec != 0 || timeout->tv_usec != 0 }),
  - select attende il tempo specificato
  - select() ritorna, se è pronto, uno (o più) dei descrittori specificati (numero positivo) oppure 0 se è scaduto il timeout

Impostazione degli insiemi

- Macro utilizzate per gestire gli insiemi di descrittori

Function	Description
`FD_SET(int fd, fd_set *set);`	Add `fd` to the `set`.
`FD_CLR(int fd, fd_set *set);`	Remove `fd` from the `set`.
`FD_ISSET(int fd, fd_set *set);`	Return true if `fd` is in the `set`.
`FD_ZERO(fd_set *set);`	Clear all entries from the `set`.

int FD_ISSET(int fd, fd_set *set);

– Al ritorno di select(), controlla se *fd* appartiene all'insieme di descrittori *set*, verificando se il bit relativo a *fd* è pari a 1 (restituisce 0 in caso negativo, un valore diverso da 0 in caso affermativo)

Select

La funzione select() rileva i descrittori pronti (non bloccati)

- Significato diverso per i tre tipi di descrittori (lettura, scrittura, exceptional conditions)

- Un descrittore è **pronto in lettura** se:
  - Nel buffer di ricezione del socket sono arrivati byte in quantità sufficiente (soglia minima per default pari a 1, modificabile con opzione del socket SO_RCVLOWAT)
  - Il peer ha chiuso il suo lato di scrittura (ricevuto FIN, leggerà fino ad EOF)
  - Si è verificato un errore sul socket
  - Se è un socket di ascolto (LISTEN) e ci sono delle connessione pendenti

- Un descrittore è **pronto in scrittura** se:
  - Nel buffer di invio del socket c'è spazio sufficiente per inviare (soglia minima per default pari a 2048 byte, modificabile con opzione del socket SO_SNDLOWAT) ed il socket è già connesso (TCP) oppure non necessita di connessione (UDP)
  - La connessione TCP è stabilita (per UDP sempre ok)
  - Il peer ha chiuso il suo lato di lettura
  - C'è errore pendente sul socket

Select

La funzione select() rileva i descrittori pronti (non bloccati)

- Significato diverso per i tre tipi di descrittori (lettura, scrittura, eccezione)

- **Exceptional conditions**:
  - In Linux, l'unica condizione per cui select() segnala fd in exceptfds è OOB TCP data
  - Out-of-band data (OOB) per TCP → SIGURG / MSG_OOB
  - Dati urgenti segnalati così
  - Di leggono con: recv(fd, buf, len, MSG_OOB);

```
if (FD_ISSET(c, &efds)) {
        char buf[1];
        printf(">>> OOB READY!\n");
        int r = recv(c, buf, sizeof(buf),
MSG_OOB);
        if (r > 0) {
            printf("OOB DATA: %c\n", buf[0]);
        } else { perror("recv OOB");}
    }
```

Esempio

```c
#include <stdio.h>
#include <sys/time.h>
#include <sys/types.h>
#include <unistd.h>
#include <sys/select.h>
#define STDIN 0  // file descriptor for standard input

int main(void)
{
    struct timeval tv;
    fd_set readfds;

    tv.tv_sec = 2;
    tv.tv_usec = 500000;

    FD_ZERO(&readfds);
    FD_SET(STDIN, &readfds);

    // don't care about writefds and exceptfds:
    select(STDIN+1, &readfds, NULL, NULL, &tv);

    if (FD_ISSET(STDIN, &readfds))
        printf("A key was pressed!\n");
    else
        printf("Timed out.\n");

    return 0;
}
```

Esempio

```
#include <stdio.h>
#include <unistd.h>
#include <errno.h>
#include <sys/types.h>
#include <sys/select.h>
#include <sys/time.h>

int main(void) {
    fd_set readfds;
    struct timeval tv;
    int nfds = STDIN_FILENO + 1;

    FD_ZERO(&readfds);
    FD_SET(STDIN_FILENO, &readfds);

    tv.tv_sec  = 2;
    tv.tv_usec = 500000;   // 2.5 s (usec: milionesimi di sec)

    int n = select(nfds, &readfds, NULL, NULL, &tv);
    if (n < 0) {
        if (errno == EINTR) { /* ripetere se necessario */ }
        perror("select"); return 1;
    } else if (n == 0) { printf("Timed out.\n");
    } else if (FD_ISSET(STDIN_FILENO, &readfds)) {
        printf("Input disponibile (tipicamente dopo Invio).\n");
        char buf[128]; // Esempio: leggere qualcosa
        ssize_t r = read(STDIN_FILENO, buf, sizeof buf);
        if (r > 0) write(STDOUT_FILENO, buf, (size_t)r);
    }
    return 0;
}
```

Multiplexing Server

Il multiplexing dell'I/O può essere usato sul server per ascoltare su più socket contemporaneamente

- Un unico processo server (iterativo) ascolta sul socket di ascolto e sui socket di connessione

Struttura generale di un server che usa select()

- riempie una struttura fd_set con i descrittori dai quali si intende leggere
- riempie una struttura fd_set con i descrittori sui quali si intende scrivere
- chiama select() ed attendere finché non avviene qualcosa
- quando select() ritorna, controlla se uno dei descrittori ha causato il ritorno. In questo caso, serve il descrittore in base al tipo di servizio offerto
- ripetere il ciclo

Multiplexing Client

Il client deve controllare due diversi descrittori in lettura

- Lo standard input, da cui legge il testo da inviare al server
- Il socket connesso con il server:
  - su cui scriverà il testo e dal quale riceverà la risposta

Multiplexing Client

Il client deve controllare due diversi descrittori in lettura
Si ricrea il set ad ogni ciclo perché **select() consuma: trasforma il set nell'insieme degli fd pronti**

```
void str_cli_echo_sel(FILE *fd, int sockfd)
{
  int               maxd, n;
  fd_set            rset;
  char              sendline[MAXLINE], recvline[MAXLINE];

  FD_ZERO(&rset);          /* inizializza a 0 il set dei descrittori in lettura */
  for ( ; ; ) {
    FD_SET(fileno(fd), &rset);  /* inserisce il descrittore del file (stdin) */
    FD_SET(sockfd, &rset);     /* inserisce il descrittore del socket */
    maxd = (fileno(fd) < sockfd) ? (sockfd + 1): (fileno(fd) + 1);
    if (select(maxd, &rset, NULL, NULL, NULL) < 0 ) { /* attende descrittore pronto
      in lettura */
      perror("errore in select");
      exit(1);
    }
```

Multiplexing Client

Il client deve controllare due diversi descrittori in lettura

```
/* Controlla se il file (stdin) è leggibile */
if (FD_ISSET(fileno(fd), &rset)) {
  if (fgets(sendline, MAXLINE, fd) == NULL)
    return; /* non vi sono dati perché si è concluso l'utilizzo del client */
  if ((writen(sockfd, sendline, strlen(sendline))) < 0) {
    fprintf(stderr, "errore in write");
    exit(1);
  }
}
```

Multiplexing Client

Il client deve controllare due diversi descrittori in lettura

```
    /* Controlla se il socket è leggibile */
    if (FD_ISSET(sockfd, &rset)) {
      if ((n = readline(sockfd, recvline, MAXLINE)) < 0) {
        fprintf(stderr, "errore in lettura");
        exit(1);
      }
      if (n == 0) {
        fprintf(stdout, "str_cli_echo_sel: il server ha chiuso la connessione");
        return;
      }
      /* Stampa su stdout */
     recvline[n] = 0;
     if (fputs(recvline, stdout) == EOF) {
        perror("errore in scrittura su stdout");
        exit(1);
      }
    }
}
```

Multiplexing Server

Echo-server serve più connessioni contemporaneamente, con un solo thread, per questo c'è l'array `client[]`. Mantiene 2 insiemi di fd: `allset` e `rset`

```
/* L'array di interi client contiene i descrittori dei socket connessi */
for (i = 0; i < FD_SETSIZE; i++)
  client[i] = -1;

FD_ZERO(&allset); /* Inizializza a zero l'insieme dei descrittori */
FD_SET(listensd, &allset); /* Inserisce il descrittore di ascolto */

for ( ; ; ) {
  rset = allset;  /* Imposta il set di descrittori per la lettura */
  /* ready è il numero di descrittori pronti */
  if ((ready = select(maxd+1, &rset, NULL, NULL, NULL)) < 0) {
    perror("errore in select");
    exit(1);
  }
  /* Se è arrivata una richiesta di connessione, il socket di ascolto
     è leggibile: viene invocata accept() e creato un socket di connessione */
  if (FD_ISSET(listensd, &rset)) {
    len = sizeof(cliaddr);
```

Multiplexing Server

```
if ((connsd = accept(listensd, (struct sockaddr *)&cliaddr, &len)) < 0) {
    perror("errore in accept");
    exit(1);
  }

  /* Inserisce il descrittore del nuovo socket nel primo posto libero di client */
  for (i=0; i<FD_SETSIZE; i++)
    if (client[i] < 0) {
      client[i] = connsd;
      break;
    }
  /* Se non ci sono posti liberi in client, errore */
  if (i == FD_SETSIZE) {
    fprintf(stderr, "errore in accept");
    exit(1);
  }
```

Multiplexing Server

ready = select(...): numero di descrittori pronti; dopo ogni gestione --ready e break quando arriva a 0

```
  /* Altrimenti inserisce connsd tra i descrittori da controllare
     ed aggiorna maxd */
  FD_SET(connsd, &allset);
  if (connsd > maxd) maxd = connsd;
  if (i > maxi) maxi = i;
  if (--ready <= 0) /* Cicla finché ci sono ancora descrittori da controllare */
   continue;
}
/* Controlla i socket attivi per controllare se sono leggibili */
for (i = 0; i <= maxi; i++) {
  if ( (socksd = client[i]) < 0 )
    /* Se il descrittore non è stato selezionato, viene saltato */
    continue;
  if (FD_ISSET(socksd, &rset)) {
 /* Se socksd è leggibile, invoca la readline */
    if ((n = readline(socksd, buff, MAXLINE)) == 0) {
      /* Se legge EOF, chiude il descrittore di connessione */
      if (close(socksd) == -1) {
```

Multiplexing Server

Rimozione client: quando recv() dà 0 o errore → close(), FD_CLR(), client[i] = -1.

```
            perror("errore in close");
            exit(1);
            }
            /* Rimuove socksd dalla lista dei socket da controllare */
          FD_CLR(socksd, &allset);
          /* Cancella socksd da client */
          client[i] = -1;
        }
        else /* echo */
            if (writen(socksd, buff, n) < 0 ) {
             fprintf(stderr, "errore in write");
             exit(1);
            }
            if (--ready <= 0) break;
       }
      }
    }
}
```

Nome Dominio

- DNS

  - Ad un host può venir assegnato un nome di dominio (*domain name*), come www.unina.it
  - Il servizio di rete chiamato DNS (*Domain Name Service*) converte i nomi di dominio in indirizzi IP
    - www.unina.it → 143.225.5.3
    - questa conversione prende il nome di “risoluzione del nome”, dall'inglese “domain name resolution”
  - Dalla shell, il comando nslookup esegue la conversione
    - nslookup <nome>
    - esempio: nslookup www.unina.it

Nome Dominio

- Recuperare l'indirizzo del dominio

```
struct addrinfo hints = {0}, *res;
hints.ai_family = AF_INET;     // solo IPv4
getaddrinfo(argv[1], NULL, &hints, &res);
struct sockaddr_in *a = (struct sockaddr_in*)res->ai_addr;
printf("IP: %s\n", inet_ntoa(a->sin_addr));
freeaddrinfo(res);
```

`getaddrinfo()` → risolve un nome in una lista di strutture `addrinfo`
`inet_ntop()` → converte indirizzi binari in stringhe leggibili

Nome Dominio

- Recuperare l'indirizzo del dominio

  getaddrinfo() è la funzione moderna e portabile per risolvere nomi simbolici (es. www.example.com) in indirizzi IP (IPv4/IPv6) e ottenere i parametri necessari per aprire una socket.

  Risolve un nome in uno o più indirizzi IP
  Supporta IPv4 e IPv6 senza differenze per il programmatore
  Restituisce una lista di struct addrinfo pronte per socket() e connect()

```
struct addrinfo hints, *res;
memset(&hints, 0, sizeof(hints));
hints.ai_socktype = SOCK_STREAM; // TCP

getaddrinfo("www.example.com", "80", &hints, &res);

// res contiene indirizzi pronti per socket() e connect()
```

Nome Dominio

- Recuperare l'indirizzo del dominio

  Utilizzo con scorrimento degli indirizzi

```
struct addrinfo hints, *res, *p;
memset(&hints, 0, sizeof(hints));
hints.ai_socktype = SOCK_STREAM;
hints.ai_family   = AF_UNSPEC; // IPv4 e IPv6

getaddrinfo("www.example.com", "80", &hints, &res);

int sock;
for (p = res; p != NULL; p = p->ai_next) {
    sock = socket(p->ai_family, p->ai_socktype, p->ai_protocol);
    if (sock < 0) continue;

    if (connect(sock, p->ai_addr, p->ai_addrlen) == 0) {
        printf("Connesso!\n"); break; }
    close(sock);
}
if (p == NULL) {
    fprintf(stderr, "Nessun indirizzo si è connesso\n"); exit(1);}

freeaddrinfo(res);
```

Nome Dominio

- Recuperare l'indirizzo del dominio

```
#define _POSIX_C_SOURCE 200112L
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <netdb.h>
#include <arpa/inet.h>
#include <unistd.h>

int main(int argc, char *argv[]) {
    if (argc != 2) {
        fprintf(stderr, "Uso: %s <hostname>\n", argv[0]);
        exit(1);
    }
    struct addrinfo hints, *res, *p;
    char host[INET6_ADDRSTRLEN];
    memset(&hints, 0, sizeof(hints));
    hints.ai_family   = AF_UNSPEC;   // IPv4 + IPv6
    hints.ai_socktype = SOCK_STREAM; // oppure 0 per qualsiasi
```

Nome Dominio

- Recuperare l'indirizzo del dominio

```
int err = getaddrinfo(argv[1], NULL, &hints, &res);
    if (err != 0) {
        fprintf(stderr, "Errore: %s\n", gai_strerror(err));
        exit(1);
    }
    for (p = res; p != NULL; p = p->ai_next) {
        void *addr;
        const char *ver;

        if (p->ai_family == AF_INET) {
            struct sockaddr_in *ipv4 = (struct sockaddr_in *)p->ai_addr;
            addr = &(ipv4->sin_addr);
            ver = "IPv4";
        } else if (p->ai_family == AF_INET6) {
            struct sockaddr_in6 *ipv6 = (struct sockaddr_in6 *)p->ai_addr;
            addr = &(ipv6->sin6_addr);
            ver = "IPv6";
        } else { continue;}
        inet_ntop(p->ai_family, addr, host, sizeof(host));
        printf("%s: %s\n", ver, host);
    }
    freeaddrinfo(res);
    return 0;
}
```